PRACTICAL NO.7

Title: PL/SQL Control Statement

1.Write a PL/SQL program to demonstrate statements (If , If-Else , Nested , Ladder)

i) If

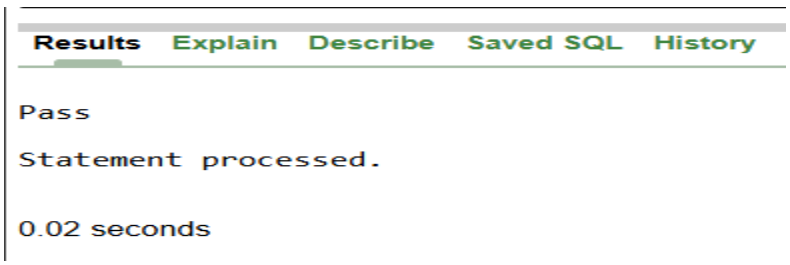
Syntax:

```
IF condition THEN
   statements;
END IF;
```

Example:

```
DECLARE
  marks NUMBER := 75;
BEGIN
  IF marks >= 40 THEN
    DBMS_OUTPUT.PUT_LINE('Pass');
  END IF;
END;
/
```

Output:



ii) If-Else

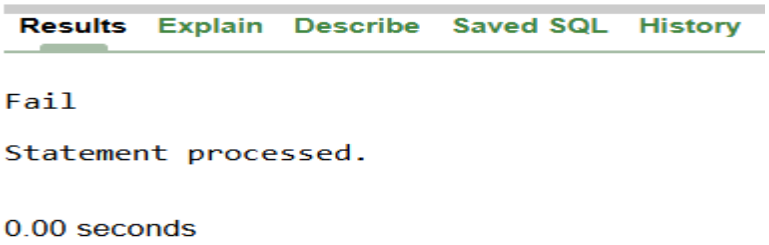
Syntax:

```
IF condition THEN
   statements;
ELSE
   statements;
END IF;
```

Example:

```
DECLARE
   marks NUMBER := 35;
BEGIN
   IF marks >= 40 THEN
      DBMS_OUTPUT.PUT_LINE('Pass');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Fail');
   END IF;
END;
/
```

Output:



iii) Nested

Syntax:

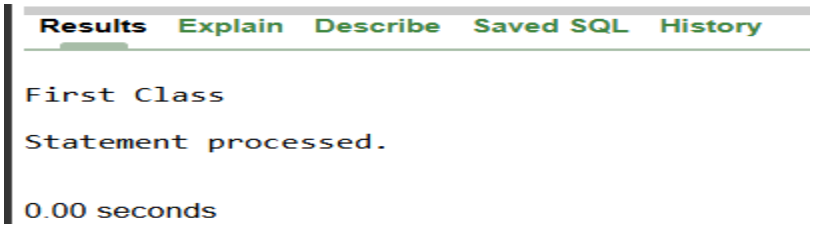
```
IF condition1 THEN
   IF condition2 THEN
      statements;
   ELSE
      statements;
   END IF;
ELSE
   statements;
END IF;
```

Example:

```
DECLARE
   marks NUMBER := 75;
```

```
BEGIN
   IF marks >= 40 THEN
      IF marks >= 60 THEN
         DBMS_OUTPUT.PUT_LINE('First Class');
      ELSE
         DBMS_OUTPUT.PUT_LINE('Pass');
      END IF;
   ELSE
      DBMS_OUTPUT.PUT_LINE('Fail');
   END IF;
END;
/
```

Output:



iv) Ladder

Syntax:

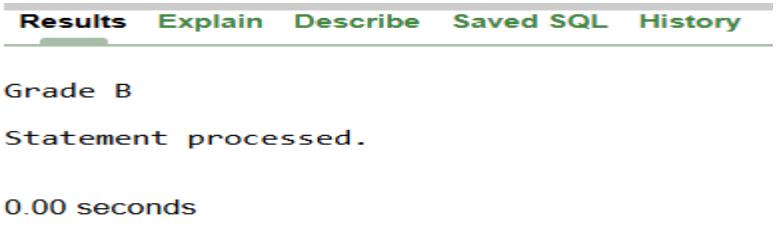
```
IF condition1 THEN
   statements;
ELSIF condition2 THEN
   statements;
ELSIF condition3 THEN
   statements;
ELSE
   statements;
END IF;
```

Example:

```
DECLARE
```

```
    marks NUMBER := 85;
BEGIN
    IF marks >= 90 THEN
        DBMS_OUTPUT.PUT_LINE('Grade A');
    ELSIF marks >= 75 THEN
        DBMS_OUTPUT.PUT_LINE('Grade B');
    ELSIF marks >= 40 THEN
        DBMS_OUTPUT.PUT_LINE('Pass');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Fail');
    END IF;
END;
/
```

Output:



2. Develop a PL/SQL program using a CASE statement to display grade based on marks.

Syntax:

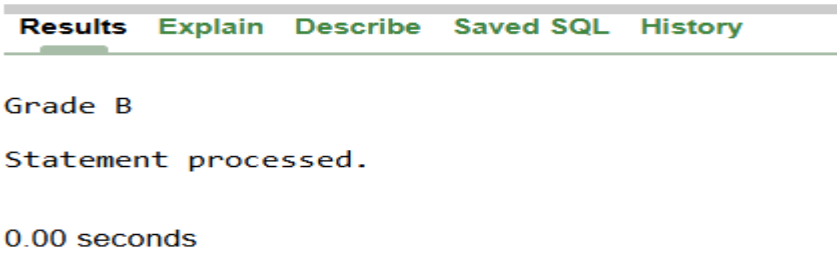
```
CASE expression
    WHEN value1 THEN
        statements;
    WHEN value2 THEN
        statements;
    ELSE
        statements;
END CASE;
```

Example:

```
DECLARE
```

```
   marks NUMBER := 85;
BEGIN
   CASE
      WHEN marks >= 90 THEN
         DBMS_OUTPUT.PUT_LINE('Grade A');
      WHEN marks >= 75 THEN
         DBMS_OUTPUT.PUT_LINE('Grade B');
      WHEN marks >= 60 THEN
         DBMS_OUTPUT.PUT_LINE('Grade C');
      WHEN marks >= 40 THEN
         DBMS_OUTPUT.PUT_LINE('Grade D');
      ELSE
         DBMS_OUTPUT.PUT_LINE('Fail');
   END CASE;
END;
/
```

Output:



3.Write PL/SQL programs using LOOP , WHILE LOOP and FOR LOOP .

i) Simple LOOP

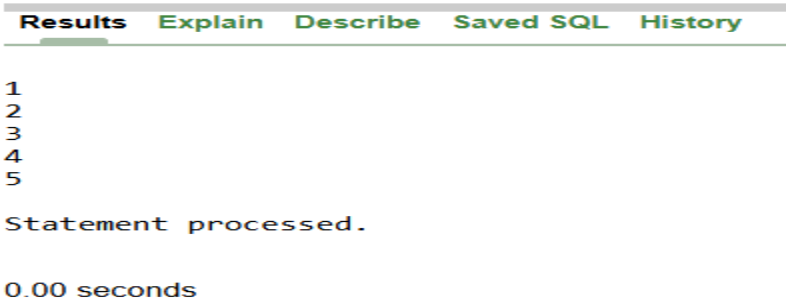
Synatx:

```
LOOP
 statements;
   EXIT WHEN condition;
END LOOP;
```

Example:

```
DECLARE
   i NUMBER := 1;
BEGIN
   LOOP
      DBMS_OUTPUT.PUT_LINE(i);
      i := i + 1;
      EXIT WHEN i > 5;
   END LOOP;
END;
/
```

Output:



ii) WHILE LOOP

Synatx:

```
WHILE condition LOOP
   statements;
END LOOP;
```

Example:

```
DECLARE
   i NUMBER := 1;
BEGIN
   WHILE i <= 7 LOOP
      DBMS_OUTPUT.PUT_LINE(i);
      i := i + 1;
   END LOOP;
```

```
END;
/
```

Output:




```
1
2
3
4
5
6
7

Statement processed.


0.00 seconds
```

iii) FOR Loop

Synatx:

```
FOR variable IN lower_bound..upper_bound LOOP
   statements;
END LOOP;
```

Example:

```
BEGIN
   FOR i IN 5..10 LOOP
      DBMS_OUTPUT.PUT_LINE(i);
   END LOOP;
END;
/
```

Output:




```
5
6
7
8
9
10

Statement processed.
```